

VISERON

AI Operating System for Autonomous Operations

System Evolution Report — arquitetura, capacidades,
de consolidação

OMEGA Kernel

E2E Task Execution

Composite Verification

Architecture Intelligence

Graphify 4.278 nós

Event Bus reativo

DATA · 08/08/2026

www.trinnityviseronssystem.io

TVS v5.0

1 Resumo executivo

O Trinnity Viseron System (TVS) é um sistema operativo de IA para organizações autónomas: objetivos entram, são planeados pelo kernel OMEGA, executados por agentes reais com ferramentas, verificados por uma engine composta e gravados na memória — nada morre no restart.

Commit: 3678e4fde1eac48980c9759a64e2efd2c5198c77

Ramo: main

Knowledge graph: 4278 nós · 8275 arestas · 282 comunidades

Testes: OMEGA 174/174 · core 14/14 · web n/a · OS n/a

2 Estado atual — consolidado (FREEZE)

O sistema encontra-se num ponto de consolidação. O refactor estrutural da Fase A fechou os quatro pilares técnicos (ciclos de import, contrato de bridges, verificação composta e inteligência de arquitetura). Segue-se o redesign da presença web para refletir a verdade da arquitetura.

% Refactor estrutural do OMEGA concluído e testado (174/174).

% Backup do site em VISERON_SITE_BACKUP/ (manifest + commit hash).

% Sem novas expansões de agentes até a capacidade real estar provada.

3 Verdade prática (o que o sistema é de facto)

O TVS não é ainda uma superinteligência de 5.000 processos independentes: o número '5.396 mentes' é uma arquitetura de definições de agentes. O que é REAL e verificável: 10 agentes nucleares, squads AIOX, pipeline E2E de tarefas com verificação, event bus reativo, memória persistente, e 4 suítes de testes verdes.

% Provar capacidade real > declarar 5.000 agentes.

% Benchmarks e casos verificáveis são o ativo mais importante.

4 Arquitetura OMEGA (o kernel)

OMEGA Kernel: runtime de eventos, estado, tarefas, memória, ferramentas, segurança e auditoria. É o cérebro operacional que os módulos de negócio usam para executar trabalho de verdade.

% EventBus — backbone reativo (wildcards, isolamento, replay, SSE/Socket.IO).

% TaskQueue — 9 estados, fila persistente, cancelamento, retoma após restart.

% Permissions — roles/grants com assert por ação.

% Kernel — facade com 4 adapters (AgentRegistry, Memory, Tools, AIRouter).

5 Execution OS — execução E2E verificada

Uma tarefa entra, recebe ID, passa por plano !' fila !' agente !' ferramentas !' execução !' verificação !' resultado, e é persistida + gravada na memória.

POST /api/omega/tasks

Criar tarefa (type + title + payload)

Estado: *CREATED !' PLANNING !' QUEUED !' RUNNING !' VERIFYING !' COMPLETED*

GET /api/omega/tasks/list?status=

Listar tarefas por estado

tasks/:id, tasks/:id/cancel, tasks/history

6 Event Bus — backbone reativo

%, Wildcards: task.* casa task.completed e sub-tópicos; * casa tudo.

%, Isolamento: um handler que falha não quebra os outros; erro em eventbus.handler.error.

%, Histórico ring buffer (500 eventos) com replay para novos subscritores.

%, EventBridge: MemoryEngine !' bus, bus !' Socket.IO, bus !' SSE.

7 Composite Verification (nova)

A verificação deixou de ser uma função ad-hoc: existe agora uma interface Verifier e um CompositeVerifier que agrega vários verificadores e calcula o estado mais grave (PASS < RETRY < HUMAN < FAIL).

%, Verifier interface + toVerifierFn (adapta para o kernel).

%, CompositeVerifier: um FAIL sobrepõe-se a RETRY; erro num verifier não abate os outros.

%, Kernel.attachVerifier(verifier) — exposto no status do kernel.

8 Architecture Intelligence (nova)

Módulo que lê o knowledge graph do repositório (Graphify) e responde a perguntas estruturais com subgrafos pequenos e relevantes — nunca o grafo inteiro.

GET /api/omega/architecture?q=TaskQueue

Subgrafo de contexto (nós + ficheiros + razão)

Provenance: VISERON · Pedro-Trinnity · AIOX · OMEGA · graphify

GET /api/omega/architecture/risks

Riscos: hubs, alto acoplamento, impacto

Risco severidade low/medium/high

GET /api/omega/architecture/path?from=A&to=B

Caminho entre dois módulos

impact?subject=&hops=

9 Integration contract (nova)

As 8 bridges do sistema (ASNO, CallSystem, N8N, OmniRouteBridge, OmniRouteHub, OpenJarvis, TVSTools, ViseronApps) implementam agora a interface comum IntegrationBridge com initialize(): Promise<number>. A SuperIntegration usa initBridge/shutdownBridge do contrato — eliminando 7 blocos duplicados.

src/integrations/contract.ts

```
IntegrationBridge { name, initialize(), stop?(), status?() }
```

8 bridges normalizadas · Promise<number>

10 Graphify — knowledge graph do repositório

Nós: 4278

Arestas: 8275

Comunidades: 282

Built at: 2f5b2e317b12d3eaa88abc77dd6094addcb97a6b

Top hubs (grau):

%, package_scripts — grau 139

%, src_core_memory_memoryengine_memoryengine — grau 109

%, src_web_standalone_server — grau 97

%, src_core_agentmanager_agentmanager — grau 84

%, tests_omega_test_runomegatests — grau 72

%, src_core_viseroncore — grau 69

Top acoplamento cross-file:

%, src/core/types/index.ts — 234 ligações cross-file

%, src/core/memory/MemoryEngine.ts — 172 ligações cross-file

%, src/web/standalone-server.ts — 157 ligações cross-file

%, src/core/ViseronCore.ts — 122 ligações cross-file

%, tests/omega.test.ts — 117 ligações cross-file

%, src/core/AgentManager.ts — 115 ligações cross-file

11 Agentes nucleares (10)

CEO, Planner, Researcher, Engineer, Operator, Finance, Sales, Security, Verifier, Evolution. Regra de escala: aperfeiçoar estes 10 e escalar 10!100!1.000!10.000 pela capacidade, não por nomes.

%, AgentRuntime: specs por manifesto, loadSpecsFromDir, registerHook.

%, AgentManager: registro, list, getAgentsByCapability, run.

12 Squads AIOX

Cinco squads de supervisão/execução sobre os agentes nucleares, carregados de manifests e operáveis via API. O watchdog self-heal registra kernel, runtime, squads, enterprise e factory.

GET /api/omega/squads

Listar squads + membros

squads/:id, squads/:id/run

13 Autonomia — ciclos de planeamento

O AutonomyLayer corre ciclos (planning/evolution/learning) que enfileiram trabalho no kernel: as mentes nunca ficam a 0 tarefas. Ciclos configuráveis via startAutonomyCycles(intervalMs).

POST /api/omega/autonomy/cycle

Disparar ciclo { kind }

planning | evolution | learning

14 Self-healing

SelfHealWatchdog monitoriza componentes registados, detecta corações parados e dispara healNow() para os recuperar. O ciclo de imports foi quebrado (import type) para o tornar independente do AgentRuntime e do SquadRegistry.

POST /api/omega/watchdog/heal

Forçar cura de um componente

watchdog · healNow(component)

15 Memória persistente

- % KnowledgeGraph: entidades, relações, tipos — ficheiro JSON persistente.
- % MemoryEngine: STM/LTM + vector store (Qdrant com fallback em memória).
- % Cada task:completed/failed grava no graph + memória de longo prazo.
- % EventBridge consolida os eventos de memória no kernel bus.

16 Segurança

- % Permissions: roles (root/commander/agent/viewer), grants com wildcard, assert por ação.
- % Governança bíblica: 9 princípios que bloqueiam fraude, mentiras e vazamento de chaves.
- % Wallets cripto: seeds nunca no chat/commits; backup obrigatório antes de sobrescrever.

17 Testes — estado real

OMEGA: 174/174
Core: 14/14
Web: n/a
TVS OS: n/a
Lint (tsc --noEmit):OK

18 API — superfície pública

GET /api/omega/status

Estado do kernel + verifier + architecture
kernel, tasks, verifier, events, architecture

GET /api/omega/events?topic=task.*

Stream SSE em tempo real
events/history para replay

POST /api/omega/tasks

Criar tarefa E2E
{ type, title, payload, priority }

19 Roteiro — próximos passos

- % Redesign do site como 'AI Operating System for Autonomous Operations' (LIVE / IN DEVELOPMENT / ROADMAP).
- % Benchmark TVS Autonomous Organization (100 tarefas reais) — npm run omega:bench.
- % 20 secções deste relatório refletidas na landing page.
- % Deploy: GitHub + Vercel + rebuild APK.

20 Autoridade e autoria

Os direitos autorais do projeto pertencem a Pedro Costa (Comandante) e Trinnity Hurtado (Rainha). Nenhuma decisão de arquitetura, domínio, receita ou publicidade é tomada sem a aprovação deles. O sistema mantém e expõe esta autoria em todos os artefactos.

Orquestrador: Pedro Costa · Trinnity Hurtado

Squad de supervisão: AIOX (AIOX-1..5 + ARKOM)

Kernel: TVS OMEGA

Fonte de verdade estrutural: Graphify (graphify-out/graph.json)

Site: www.trinnityviseronssystem.io